Export SQL Server Table to Parquet Using Python

Overview

This document outlines the process of exporting SQL Server table data into Parquet files using a Python script with parallel processing. The solution handles large datasets efficiently by chunking data based on a primary key and writing each batch to a separate Parquet file.

Prerequisites

System Requirements

- Python 3.9+
- SQL Server ODBC Driver 17

Python Setup

1. Create and activate a virtual environment: Open a pwershell scrip window and enterted the following:

```
py --version
python -m venv venv
B:\Adeel Syed\Python\venv> .\venv\Scripts\Activate.ps1
```

2. Allow running local scripts

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

3. Install Required Libraries:

```
pip install -r requirements.txt
```

File: - requirements.txt

```
greenlet==3.2.4
numpy==2.3.3
pandas==2.3.3
pyarrow==21.0.0
pyodbc==5.2.0
python-dateutil==2.9.0.post0
pytz==2025.2
six==1.17.0
SQLAlchemy==2.0.43
typing_extensions==4.15.0
tzdata==2025.2
```

Script Description

Main Components

- create_connection(): Returns SQLAlchemy connection string

- get_table_schema(): Reads SQL Server schema and maps types to pandas dtypes
- export_batch(): Fetches one batch and writes it to a `.parquet` file
- main(): Handles argument parsing, batching logic, logging, and parallel execution

Output Directory Structure

```
<os_path>/
  └── <server>/
      └── <database>/
          └── dbo.<table>/
              ├── <table>_part_1.parquet
              ├── <table>_part_2.parquet
              └── ...
          └── <table>_log.txt
```

Scripts Arguments

Argument	Description	Example
server	SQL Server hostname	mydbserver123
database	SQL Server DB name	SalesDB
table	Table to export	Orders
keycolumn	Primary key column (optional)	OrderID
processes	No. of Parallel processes (optional)	2
os_path	Output base path (optional)	B:\DataExport\Parquet

Example Execution

```
python "B:\Adeel Syed\Python\venv\parquet_export_multi_v1.py" tdb19bscax2012 INVENTJOURNALTABLE
```

If the table does not have a primary key, or if you're extracting data from a view, you must manually specify a key column. Identify an appropriate indexed or unique column and pass it

explicitly as shown below.

```
python "B:\Adeel Syed\Python\venv\parquet_export_multi_v1.py"
TDB08AX2012 StoreOrders AXInvoiceDetails --keycolumn ITEMID
```

Sample output

```
Extract of inventjournaltable started at 2025-07-01 16:45:30
Server: tdb19b, Database: scax2012, Table: INVENTJOURNALTABLE,
Processes: 1
--------------------------------------------------------------------------------
------------------------------
Processed: inventjournaltable_part_1.parquet | Batch Range: 5637145330
- 5637281207 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_2.parquet | Batch Range: 5637281208
- 5637393524 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_3.parquet | Batch Range: 5637393525
- 5637511641 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_4.parquet | Batch Range: 5637511642
- 5637630037 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_5.parquet | Batch Range: 5637630038
- 5637763400 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_6.parquet | Batch Range: 5637763401
- 5637889687 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_7.parquet | Batch Range: 5637889690
- 5638021381 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_8.parquet | Batch Range: 5638021382
- 5638180408 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_9.parquet | Batch Range: 5638180409
- 5638333785 | Rows: 100000 | Duration: 00:00:04
Processed: inventjournaltable_part_10.parquet | Batch Range:
5638333786 - 5638442702 | Rows: 77585 | Duration: 00:00:03
--------------------------------------------------------------------------------
------------------------------
Process ended at 2025-07-01 16:46:15

Extract completed successfully. Total Rows Processed: 977585 | Total
Duration: 00:00:45 (hh:mm:ss)
```

Python Script: - parquet_export_multi_v1.py

```python
import pandas as pd
from sqlalchemy import create_engine, text
import os
import time
import argparse
from multiprocessing import Pool

def create_connection(server_name, database_name):
    conn_str = f'mssql+pyodbc:///?odbc_connect=DRIVER={{ODBC Driver
17 for SQL Server}};SERVER={server_name};DATABASE=
{database_name};Trusted_Connection=yes;'
    return conn_str

def get_table_schema(engine, table_name):
    query = f"""
    SELECT COLUMN_NAME, DATA_TYPE
    FROM INFORMATION_SCHEMA.COLUMNS
    WHERE TABLE_NAME = '{table_name}'
    """
```

```
    schema_df = pd.read_sql(query, engine)
    dtype_mapping = {
        "int": "Int32", "bigint": "Int64", "smallint": "Int16", "tinyint": "Int8", "bit": "boolean",
        "decimal": "float64", "numeric": "float64", "money": "float64", "smallmoney": "float64",
        "float": "float64", "real": "float32", "datetime": "datetime64[ns]", "smalldatetime": "datetime64[ns]",
        "date": "datetime64[ns]", "time": "timedelta64[ns]", "char": "string", "nchar": "string",
        "nvarchar": "string", "varchar": "string", "text": "string", "ntext": "string"
    }
    return {row.COLUMN_NAME: dtype_mapping.get(row.DATA_TYPE, 'string') for _, row in schema_df.iterrows()}

def get_primary_key(engine, table_name):
    query = f"""
    SELECT COLUMN_NAME
    FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
    WHERE OBJECTPROPERTY(OBJECT_ID(CONSTRAINT_SCHEMA + '.' + QUOTENAME(CONSTRAINT_NAME)), 'IsPrimaryKey') = 1
    AND TABLE_NAME = '{table_name}'
    """
    result = pd.read_sql(query, engine)
    if not result.empty:
        return result.iloc[0]["COLUMN_NAME"]
    else:
        raise ValueError(f"No primary key found for table: {table_name}")

def export_batch(args):
    server_name, database_name, table_name, primary_key_column, batch_range, output_dir, schema_dtypes, log_file = args
    conn_str = create_connection(server_name, database_name)
    engine = create_engine(conn_str)
    start, end, file_count = batch_range

    start_time = time.time()
    query = f"SELECT * FROM {table_name} WITH (NOLOCK) WHERE {primary_key_column} BETWEEN {start} AND {end}"
    df = pd.read_sql(text(query), engine)

    for col, dtype in schema_dtypes.items():
        try:
            if dtype == "datetime64[ns]":
                df[col] = pd.to_datetime(df[col], errors="coerce")  # Replace invalid datetimes with NaT
            else:
                df[col] = df[col].astype(dtype)
        except Exception as e:
            print(f"Warning: Failed to convert column '{col}' to {dtype}. Error: {e}")
            df[col] = None

    file_name = f"{table_name}_part_{file_count}.parquet"
    parquet_file = os.path.join(output_dir, file_name)
    df.to_parquet(parquet_file, engine='pyarrow', index=False)

    duration = time.time() - start_time
    log_entry = f"Processed: {file_name} | Batch Range: {start} - {end} | Rows: {len(df)} | Duration: {time.strftime('%H:%M:%S', time.gmtime(duration))}\n"

    with open(log_file, "a") as log:
        log.write(log_entry)
        log.flush()

    print(log_entry.strip())
    return len(df)

def main():
```

```
    parser = argparse.ArgumentParser(description="Export table data to Parquet with parallel processing.")
    parser.add_argument("server")
    parser.add_argument("database")
    parser.add_argument("table")
    parser.add_argument("--keycolumn", help="Optional: Primary key column for batching. Auto-detected if not passed.")
    parser.add_argument("--processes", type=int, choices=[1, 2, 4], default=1, help="Number of parallel processes")
    parser.add_argument("--os_path", default="B:\\Adeel Syed\\AzureDevops\\Sigma\\Parquet", help="Output directory base path")
    args = parser.parse_args()

    server_name = args.server.lower()
    database_name = args.database.lower()
    table_name = args.table.lower()
    os_path = args.os_path
    processes = args.processes

    conn_str = create_connection(server_name, database_name)
    engine = create_engine(conn_str)

    # Auto-detect PK if not provided
    if args.keycolumn:
        primary_key_column = args.keycolumn.lower()
    else:
        primary_key_column = get_primary_key(engine, table_name).lower()
        print(f"[INFO] Primary key auto-detected: {primary_key_column}")

    output_path = os.path.join(os_path, server_name, database_name)
    output_dir = os.path.join(output_path, f"dbo.{table_name}")
    log_file = os.path.join(output_path, f"{table_name}_log.txt")
    os.makedirs(output_dir, exist_ok=True)
    rows_per_file = 500000

    schema_dtypes = get_table_schema(engine, table_name)

    batch_query = f"""
    WITH RowNumbers AS (
        SELECT ROW_NUMBER() OVER (ORDER BY {primary_key_column}) AS RowNum, {primary_key_column}
        FROM {table_name} WITH (NOLOCK)
    ),
    GroupedRows AS (
        SELECT MIN({primary_key_column}) AS FirstRowNum, MAX({primary_key_column}) AS LastRowNum,
               (RowNum - 1) / {rows_per_file} AS GroupNumber
        FROM RowNumbers
        GROUP BY (RowNum - 1) / {rows_per_file}
    )
    SELECT FirstRowNum, LastRowNum FROM GroupedRows ORDER BY FirstRowNum;
    """

    batch_ranges = pd.read_sql(text(batch_query), engine)
    batches = [(server_name, database_name, table_name, primary_key_column,
                (row['FirstRowNum'], row['LastRowNum'], idx + 1),
                output_dir, schema_dtypes, log_file)
               for idx, row in batch_ranges.iterrows()]

    start_script = time.time()
    with open(log_file, "a") as log:
        log.write(f"Extract of {table_name} started at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_script))}\n")
        log.write(f"Server: {server_name}, Database: {database_name}, Table: {table_name}, Processes: {processes}\n")
        log.write("-" * 100 + "\n")
```

```
134
135     with Pool(processes) as pool:
136         results = pool.map(export_batch, batches)
137
138     total_rows = sum(results)
139     end_script = time.time()
140     total_duration = time.strftime('%H:%M:%S', time.gmtime(end_script
    - start_script))
141
142     final_entry = f"\nExtract completed successfully. Total Rows
    Processed: {total_rows} | Total Duration: {total_duration}
    (hh:mm:ss)\n"
143     print(final_entry.strip())
144
145     with open(log_file, "a") as log:
146         log.write("-" * 100 + "\n")
147         log.write(f"Process ended at {time.strftime('%Y-%m-%d
    %H:%M:%S', time.localtime(end_script))}\n\n")
148         log.write(final_entry)
149
150 if __name__ == "__main__":
151     main()
152
```

How It Works

1. Schema Detection: Maps SQL Server datatypes to pandas-compatible types.
2. Batching: Splits data into ranges using ROW_NUMBER() and primary key.
3. Parallel Export: Each process handles a chunk and writes to `.parquet`.
4. Logging: Every batch is logged with range, row count, and duration.

Read/Verify the Parquet extracts

File:- ReadParquet.py

```python
import pandas as pd
import argparse
import os

def main():
    # Set up argument parsing
    parser = argparse.ArgumentParser(description="Preview and export a
Parquet file")
    parser.add_argument("file_path", help="Path to the Parquet file")
    parser.add_argument("--lines", type=int, default=5, help="Number
of rows to display/export (default: 5)")
    args = parser.parse_args()

    file_path = args.file_path
    num_lines = args.lines

    # Check if file exists
    if not os.path.exists(file_path):
        print(f"File not found: {file_path}")
        return

    # Extract base name for output files
    base_name = os.path.splitext(os.path.basename(file_path))[0]
    data_csv = f"{base_name}_data_rows.csv"
    dtypes_csv = f"{base_name}_data_columns.csv"

    # Read Parquet file
```

```
    print(f"Reading Parquet file: {file_path}")
    df = pd.read_parquet(file_path)

    # Print basic info
    print(f"Total rows: {len(df)}")
    print(f"Total columns: {len(df.columns)}\n")

    # Preview rows
    print(f"Displaying first {num_lines} rows:\n")
    preview_df = df.head(num_lines)
    print(preview_df)

    # Export preview to CSV
    preview_df.to_csv(data_csv, index=False)
    print(f"\nFirst {num_lines} rows exported to: {data_csv}")

    # Export column data types to CSV
    dtypes_df = pd.DataFrame(df.dtypes, columns=["dtype"])
    dtypes_df.index.name = "column_name"
    dtypes_df.reset_index(inplace=True)
    dtypes_df.to_csv(dtypes_csv, index=False)

    # Print dtypes
    print(f"\n Column data types:")
    with pd.option_context('display.max_rows', None,
'display.max_columns', None):
        print(dtypes_df)

    print(f"\nColumn data types exported to: {dtypes_csv}")

if __name__ == "__main__":
    main()

```

Execute

```
python .\ReadParquet.py "B:\Adeel
Syed\AzureDevops\Sigma\Parquet\tdb08ax2012\cwmgtstoreinvoices\dbo.pharm
xinvoiceitems3v0\pharmxinvoiceitems3v0_part_1.parquet"
```

This will generate 2 separate files

1. First N rows from the files (Filename_data_rows.csv)
2. Column names with their data types (Filename_data_columns.csv)

The Parquet extract Python script has been enhanced to support both *Key column* and *Timestamp column* modes. You can choose whichever column is indexed to improve extraction performance.

- **--keycolumn** → Extracts data in batches based on primary key ranges.
- **--timecolumn** → Extracts data in daily timestamp ranges, generating one Parquet file per day.

```
import pandas as pd
from sqlalchemy import create_engine, text
import os
import time

```

```
import argparse
from multiprocessing import Pool

def create_connection(server_name, database_name):
    conn_str = f'mssql+pyodbc:///?odbc_connect=DRIVER={{ODBC Driver 17 for SQL Server}};SERVER={server_name};DATABASE={database_name};Trusted_Connection=yes;'
    return conn_str

def get_table_schema(engine, table_name):
    query = f"""
    SELECT COLUMN_NAME, DATA_TYPE
    FROM INFORMATION_SCHEMA.COLUMNS
    WHERE TABLE_NAME = '{table_name}'
    """
    schema_df = pd.read_sql(query, engine)
    dtype_mapping = {
        "int": "Int32", "bigint": "Int64", "smallint": "Int16", "tinyint": "Int8", "bit": "boolean",
        "decimal": "float64", "numeric": "float64", "money": "float64", "smallmoney": "float64",
        "float": "float64", "real": "float32", "datetime": "datetime64[ns]", "smalldatetime": "datetime64[ns]",
        "date": "datetime64[ns]", "time": "timedelta64[ns]", "char": "string", "nchar": "string",
        "nvarchar": "string", "varchar": "string", "text": "string", "ntext": "string"
    }
    return {row.COLUMN_NAME: dtype_mapping.get(row.DATA_TYPE, 'string') for _, row in schema_df.iterrows()}

def get_primary_key(engine, table_name):
    query = f"""
    SELECT COLUMN_NAME
    FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
    WHERE OBJECTPROPERTY(OBJECT_ID(CONSTRAINT_SCHEMA + '.' + QUOTENAME(CONSTRAINT_NAME)), 'IsPrimaryKey') = 1
    AND TABLE_NAME = '{table_name}'
    """
    result = pd.read_sql(query, engine)
    if not result.empty:
        return result.iloc[0]["COLUMN_NAME"]
    else:
        raise ValueError(f"No primary key found for table: {table_name}")

def export_batch(args):
    server_name, database_name, table_name, column_name, batch_range, output_dir, schema_dtypes, log_file, mode = args
    conn_str = create_connection(server_name, database_name)
    engine = create_engine(conn_str)
    start, end, file_tag = batch_range

    start_time = time.time()

    # --- Option 2: truncate timestamps to milliseconds ---
    if mode == "time":
        start_str = pd.to_datetime(start).strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        end_str   = pd.to_datetime(end).strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        query = f"""
        SELECT * FROM {table_name} WITH (NOLOCK)
        WHERE {column_name} >= '{start_str}' AND {column_name} < '{end_str}'
        """
    else:  # pk mode
        query = f"""
        SELECT * FROM {table_name} WITH (NOLOCK)
        WHERE {column_name} BETWEEN {start} AND {end}
        """

```

```
    df = pd.read_sql(text(query), engine)

    for col, dtype in schema_dtypes.items():
        try:
            if dtype == "datetime64[ns]":
                df[col] = pd.to_datetime(df[col], errors="coerce")
            else:
                df[col] = df[col].astype(dtype)
        except Exception as e:
            print(f"Warning: Failed to convert column '{col}' to {dtype}. Error: {e}")
            df[col] = None

    if mode == "time":
        file_name = f"{table_name}_{file_tag}.parquet"
    else:
        file_name = f"{table_name}_part_{file_tag}.parquet"

    parquet_file = os.path.join(output_dir, file_name)
    df.to_parquet(parquet_file, engine="pyarrow", index=False)

    duration = time.time() - start_time
    log_entry = f"Processed: {file_name} | Rows: {len(df)} | Duration: {time.strftime('%H:%M:%S', time.gmtime(duration))}\n"

    with open(log_file, "a") as log:
        log.write(log_entry)
        log.flush()

    print(log_entry.strip())
    return len(df)

def export_null_batches(server_name, database_name, table_name, output_dir, schema_dtypes, log_file, batch_size=500000):
    """Export rows with NULL timestamp in batches"""
    conn_str = create_connection(server_name, database_name)
    engine = create_engine(conn_str)
    offset = 0
    idx = 0
    while True:
        query = f"""
            SELECT * FROM {table_name} WITH (NOLOCK)
            WHERE timestamp IS NULL
            ORDER BY (SELECT NULL)
            OFFSET {offset} ROWS FETCH NEXT {batch_size} ROWS ONLY
        """
        df = pd.read_sql(text(query), engine)
        if df.empty:
            break
        file_name = f"{table_name}_null_batch_{idx}.parquet"
        parquet_file = os.path.join(output_dir, file_name)
        df.to_parquet(parquet_file, engine="pyarrow", index=False)
        duration = len(df)
        log_entry = f"Processed NULL batch: {file_name} | Rows: {len(df)}\n"
        with open(log_file, "a") as log:
            log.write(log_entry)
            log.flush()
        print(log_entry.strip())
        idx += 1
        offset += batch_size

def main():
    parser = argparse.ArgumentParser(description="Export table data to Parquet with parallel processing.")
    parser.add_argument("server")
    parser.add_argument("database")
    parser.add_argument("table")
    parser.add_argument("--keycolumn", help="Optional: Primary key column for batching.")
    parser.add_argument("--timecolumn", help="Optional: Timestamp column for batching by day.")
```

```
    parser.add_argument("--processes", type=int, choices=[1, 2, 4], default=1, help="Number of parallel processes")
    parser.add_argument("--os_path", default="B:\\Adeel Syed\\AzureDevops\\Sigma\\Parquet", help="Output directory base path")
    args = parser.parse_args()

    server_name = args.server.lower()
    database_name = args.database.lower()
    table_name = args.table.lower()
    os_path = args.os_path
    processes = args.processes

    conn_str = create_connection(server_name, database_name)
    engine = create_engine(conn_str)

    # Decide mode
    if args.timecolumn:
        column_name = args.timecolumn.lower()
        mode = "time"
    else:
        if args.keycolumn:
            column_name = args.keycolumn.lower()
        else:
            column_name = get_primary_key(engine, table_name).lower()
            print(f"[INFO] Primary key auto-detected: {column_name}")
        mode = "pk"

    output_path = os.path.join(os_path, server_name, database_name)
    output_dir = os.path.join(output_path, f"dbo.{table_name}")
    log_file = os.path.join(output_path, f"{table_name}_log.txt")
    os.makedirs(output_dir, exist_ok=True)

    schema_dtypes = get_table_schema(engine, table_name)

    if mode == "time":
        # build batches by day starting after 2025-04-16
        batch_query = f"""
        SELECT
            CAST({column_name} AS DATE) AS BatchDate,
            MIN({column_name}) AS StartTime,
            DATEADD(DAY, 1, CAST({column_name} AS DATE)) AS EndTime
        FROM {table_name} WITH (NOLOCK)
        --WHERE {column_name} >= '2025-04-17'
        GROUP BY CAST({column_name} AS DATE)
        ORDER BY BatchDate
        """
        batch_ranges = pd.read_sql(text(batch_query), engine)
        batches = [(server_name, database_name, table_name, column_name,
                    (row['StartTime'], row['EndTime'], str(row['BatchDate'])),
                    output_dir, schema_dtypes, log_file, mode)
                   for _, row in batch_ranges.iterrows()]
    else:
        rows_per_file = 500000
        batch_query = f"""
        WITH RowNumbers AS (
            SELECT ROW_NUMBER() OVER (ORDER BY {column_name}) AS RowNum, {column_name}
            FROM {table_name} WITH (NOLOCK)
            --WHERE {column_name} > '2025-04-16'
        ),
        GroupedRows AS (
            SELECT MIN({column_name}) AS FirstRowNum, MAX({column_name}) AS LastRowNum,
                   (RowNum - 1) / {rows_per_file} AS GroupNumber
            FROM RowNumbers
            GROUP BY (RowNum - 1) / {rows_per_file}
        )
        SELECT FirstRowNum, LastRowNum FROM GroupedRows ORDER BY FirstRowNum;
```

```
        """
        batch_ranges = pd.read_sql(text(batch_query), engine)
        batches = [(server_name, database_name, table_name,
column_name,
                    (row['FirstRowNum'], row['LastRowNum'], idx + 1),
                    output_dir, schema_dtypes, log_file, mode)
                   for idx, row in batch_ranges.iterrows()]

    # start extraction
    start_script = time.time()
    with open(log_file, "a") as log:
        log.write(f"Extract of {table_name} started at
{time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(start_script))}\n")
        log.write(f"Server: {server_name}, Database: {database_name},
Table: {table_name}, Processes: {processes}, Mode: {mode}\n")
        log.write("-" * 100 + "\n")

    with Pool(processes) as pool:
        results = pool.map(export_batch, batches)

    total_rows = sum(results)

    # Export NULL timestamp batches
    export_null_batches(server_name, database_name, table_name,
output_dir, schema_dtypes, log_file)

    end_script = time.time()
    total_duration = time.strftime('%H:%M:%S', time.gmtime(end_script
- start_script))

    final_entry = f"\nExtract completed successfully. Total Rows
Processed: {total_rows} | Total Duration: {total_duration}
(hh:mm:ss)\n"
    print(final_entry.strip())

    with open(log_file, "a") as log:
        log.write("-" * 100 + "\n")
        log.write(f"Process ended at {time.strftime('%Y-%m-%d
%H:%M:%S', time.localtime(end_script))}\n\n")
        log.write(final_entry)

if __name__ == "__main__":
    main()

```

Execute:

```
python parquet_export_multi_timestamp_null.py tdb14 transactionStorage
TransactionAuditSaleActivityLog --keycolumn SalesActivityLogID

python parquet_export_multi_timestamp_null.py tdb14 transactionStorage
TransactionAuditSaleActivityLog --timecolumn Timestamp
```

Stop/Kill the Parquet Extract Process using PowerShell

When a Parquet extract job is running in the background, it spawns a Python process. If the job hangs or needs to be forcefully stopped, follow the steps below to identify and terminate the process

1. Find the Python Process

```
PS C:\Users\adeel.syed> Get-Process | Where-Object {$_.ProcessName -
like "python*"}
```

Example Output

```
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    546      65   212520     139592      81.56  14936   1 python
    336      46   172856     104460       1.80  20628   1 python
```

2. Stop the Process

```
PS C:\Users\adeel.syed> Stop-Process -Id 14936, 20628 -Force
```

Extract Parquet from View

To extract parquet from the script shared by Amit. View vw_POSTransactionData is created on PBI05.TransactionsArchive and extract parquet monthwise.

```
import pandas as pd
from sqlalchemy import create_engine, text
import os
import time
import argparse

def create_connection(server_name, database_name):
    conn_str = f'mssql+pyodbc:///?odbc_connect=DRIVER={{ODBC Driver 17
for SQL Server}};SERVER={server_name};DATABASE=
{database_name};Trusted_Connection=yes;'
    return conn_str

def export_view_to_parquet(engine, view_name, timestamp_col,
output_dir, log_file=None):
    os.makedirs(output_dir, exist_ok=True)

    total_rows = 0
    start_time = time.time()

    # Step 1: Get distinct year/month values
    query_periods = f"""
    SELECT DISTINCT YEAR({timestamp_col}) AS Year,
MONTH({timestamp_col}) AS Month
    FROM {view_name} WITH (NOLOCK) ORDER BY Year, Month;
    """
    periods = pd.read_sql(text(query_periods), engine)
    script_start_time = time.time()
    # Step 2: Loop through each year-month
    for _, row in periods.iterrows():
        year, month = int(row["Year"]), int(row["Month"])

        sql = f"""
        SELECT * FROM {view_name} WITH (NOLOCK)
        WHERE YEAR({timestamp_col}) = {year}
          AND MONTH({timestamp_col}) = {month};
        """

        file_start_time = time.time()  # Start timer for this file
        df = pd.read_sql(text(sql), engine)

        parquet_file = os.path.join(output_dir, f"{view_name}_{year}-
{month:02d}.parquet")
        df.to_parquet(parquet_file, engine="pyarrow", index=False)

```

```
        # Duration for this file only
        duration = time.strftime('%H:%M:%S', time.gmtime(time.time() - file_start_time))
        total_rows += len(df)

        log_entry = f"Exported {parquet_file} | Rows: {len(df)} | Duration: {duration}\n"

        print(log_entry.strip())
        if log_file:
            with open(log_file, "a") as log:
                log.write(log_entry)
                log.flush()

    total_duration = time.strftime('%H:%M:%S', time.gmtime(time.time() - script_start_time))
    final_entry = f"\nExport complete. Files created: {len(periods)}, Total Rows: {total_rows}, Total Duration: {total_duration}\n"
    print(final_entry.strip())
    if log_file:
        with open(log_file, "a") as log:
            log.write(final_entry)

def main():
    parser = argparse.ArgumentParser(description="Export SQL view or table to Parquet, partitioned by month-year.")
    parser.add_argument("server")
    parser.add_argument("database")
    parser.add_argument("view", help="SQL view or table name")
    parser.add_argument("timestamp_col", help="Timestamp column to partition by (e.g., inTimeStamp)")
    parser.add_argument("--os_path", default="B:\\Adeel Syed\\AzureDevops\\IQVIA\\Parquet\\test", help="Output directory base path")
    args = parser.parse_args()

    conn_str = create_connection(args.server, args.database)
    engine = create_engine(conn_str)

    output_path = os.path.join(args.os_path, args.server.lower(), args.database.lower())
    os.makedirs(output_path, exist_ok=True)
    log_file = os.path.join(output_path, f"{args.view}_log.txt")

    with open(log_file, "a") as log:
        log.write(f"Export started at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime())}\n")
        log.write(f"Server: {args.server}, Database: {args.database}, View: {args.view}, Timestamp: {args.timestamp_col}\n")
        log.write("-" * 100 + "\n")

    export_view_to_parquet(engine, args.view, args.timestamp_col, output_path, log_file)

if __name__ == "__main__":
    main()

```